

GMSH for FEM: Project Report

Table of Contents

- **1. Introduction and Project Context**
 - 1.1 Purpose of GMSH and Scripting Rationale
 - 1.2 Project Requirements and Drivers
- **2. Geometry Construction: Design and Rationale**
 - 2.1 Kernel Selection and Setup
 - 2.2 Deep Dive: Deconstructing Complex Geo Scripts
 - 2.3 Key Operations: Fragment vs. Union
 - 2.4 Critical Step: Synchronization
- **3. Physical Groups: FEM Semantics**
 - 3.1 Purpose and Export Control
 - 3.2 Commands and Naming
- **4. Meshing Workflow and Quality Control**
 - 4.1 Element Order and Generation
 - 4.2 Visualization
 - 4.3 Quality Control and Optimization
- **5. Abaqus Export and Post-Processing Imperative**
 - 5.1 Export Settings and Limitations
 - **5.2 THE AB AQU S FORMAT PROBLEM**
 - 5.3 Python Post-Processing Solution
- **6. Example Workflow: Geo Script vs. Python API**
 - 6.1 Simple Cube Example: Geo Script Approach
 - 6.2 Python API Logic for Conformal Meshes
- **7. Project Summary and Lessons Learned**

1. Introduction and Project Context

1.1 Purpose of GMSH and Scripting Rationale

GMSH is a powerful 3D mesh generator used to produce organized, high-quality, solver-ready outputs. Scripting is essential because it guarantees reproducibility and allows for automation of complex checks that are necessary for reliable FEM results.

1.2 Project Requirements and Drivers

The workflow is strictly designed to meet the Abaqus solver's requirements:

- **Conformal Mesh:** Element connectivity must be perfect across material interfaces.
 - **Physical Groups:** Groups must correctly map to materials and boundary conditions.
 - **Limitation:** The generic GMSH output file requires a final Python post-processing step to be fully solver-ready.
-

2. Geometry Construction: Design and Rationale

2.1 Kernel Selection and Setup

- **Kernel Selection (Geo Script):** The **OpenCASCADE** kernel is necessary for robust 3D modeling and complex Boolean operations. It enables advanced commands like `Extrude` and `BooleanUnion`.
- `SetFactory("OpenCASCADE");`
- **GUI Fix (Manual):** To see solid geometry while scripting, you must manually change the setting. This prevents the geometry from appearing as an unreadable hollow wireframe. The required command chain is: **Open GMSH Software in GUI → Tools → Options → Geometry → Aspect → Surface Display → Solid.**

2.2 Deep Dive: Deconstructing Complex Geo Scripts

Complex geometry is built layer-by-layer using primitives (Cylinder, Cone) and topological operations.

- **Primitives (Geo Script):** Define the base shapes.
- `Cylinder(1) = {0, 0, 0, 5, 0, 0, 0.6};` // Defines the Main Fuselage cylinder
- `Cone(2) = {5, 0, 0, 1.2, 0, 0, 0.6, 0.0};` // Defines the Nose cone
- **Extrusion and Rotation (Geo Script):** Converts a 2D surface into a 3D volume and aligns it.
- `Plane Surface(21) = {20};` // Defines the 2D surface

- `outWing[] = Extrude {0, 0, 0.4} { Surface{21}; }; // Sweeps surface 0.4 units along Z`
- `Rotate {{1, 0, 0}, {2.8, 0, 0.0}, 5*Pi/180} { Volume{wing}; } //`
Rotates the volume (Wing) around the X-axis

2.3 Key Operations: Fragment vs. Union

- **Design Lesson:** The `fragment` operation is mandatory for **multi-material FEM meshes**. It keeps volumes separate but ensures the shared faces are perfect, guaranteeing a conformal mesh.
- **Union (Geo Script):** `BooleanUnion{ Volume{...}; }` Merges multiple volumes into one (e.g., merging the fuselage parts).

2.4 Critical Step: Synchronization

- **Command (Python API):** `gmsh.model.occ.synchronize()`
- **Rationale:** This command is mandatory after any geometric edit (like `fragment`). It updates the internal GMSH model, making the new entities available for meshing and grouping. Skipping it causes silent failures.

3. Physical Groups: FEM Semantics

3.1 Purpose and Export Control

- **Rationale:** Physical groups translate geometric shapes into FEM roles (materials, boundaries).
- **Rule:** Only entities assigned to a Physical Group are exported to Abaqus.

3.2 Commands and Naming

- **Geo Script:** Defines groups directly using tag numbers.
- `Physical Volume ("Middle Cube") = {3}; // Assigns Material`
- `Physical Surface("ExternalFaces") = {1, 3, 4, ...}; // Boundary condition assignment`
- **Python API:** Used when groups are identified dynamically (e.g., using center-of-mass checks).

Python

```
gmsh.model.addPhysicalGroup(3, middle_volumes, tag=1) // Grouping 3D volumes
gmsh.model.setPhysicalName(3, 1, "MiddleCube")
```

4. Meshing Workflow and Quality Control

4.1 Element Order and Generation

- **Element Order (Configuration):** Sets the element type to linear tetrahedra (C3D4).
- `Mesh.ElementOrder = 1;`
- **Generation (Python API):** GMSH generates the mesh in sequence.

Python

```
gmsh.model.mesh.generate(2) // Generate 2D surface mesh
gmsh.model.mesh.generate(3) // Generate 3D volume mesh
```

Note: The 3D mesher creates the volume elements (tetrahedra) for the regions, such as the "Middle Cube" group defined in the previous section.

4.2 Visualization

- **Command (Python API):** Launches the GMSH GUI for visual inspection.

Python

```
gmsh.fltk.run()
```

4.3 Quality Control and Optimization

- **Optimization (Python API):** `gmsh.model.mesh.optimize('Netgen')` improves element shapes, reducing skewed elements that cause solver errors.

5. Abaqus Export and Post-Processing Imperative

5.1 Export Settings and Limitations

- **GMSH Limitation:** The raw output is generic: a single, global `*ELEMENT` list with separate `ELSET` references. GMSH can list all grouped elements in the form of `ELSETs`, but it cannot group them under separate `*ELEMENT` blocks.
- **Configuration (Settings):** These rules limit export to only necessary physical groups.
- `Mesh.SaveAll = 0;` // Only export physical groups
- `Mesh.SaveGroupsOfElements = 1;` // Ensure `ELSETs` are saved
- `Mesh.SaveGroupsOfNodes = 0;` // Avoid unnecessary Node Sets (`NSETs`)

5.2 THE ABAQUS FORMAT PROBLEM

The FEM solver requires elements to be grouped under specific `*ELEMENT` headers, which GMSH does not natively produce.

- **Solver Requirement:** The file needs separate, grouped blocks for each material/boundary.
- **Required Format for Volume Elements (C3D4):** (From the example code located in this directory)
 - `*ELEMENT, TYPE=C3D4, ELSET=MiddleCube`
 - Element ID, Node 1 ID, Node 2 ID, Node 3 ID, Node 4 ID
 - 1, 267, 270, 222, 418
 - ...
- **Required Format for Surface Elements (CPS3/CPE3):** (From the example code located in this directory)
 - `*ELEMENT, TYPE=CPS3, ELSET=ExternalFaces`
 - Element ID, Node 1 ID, Node 2 ID, Node 3 ID
 - 1, 10, 20, 30
 - ...

5.3 Python Post-Processing Solution

- **Necessity:** A separate **Python script** is **mandatory** to take the raw GMSH output, perform file handling to match element IDs with their `ELSETs`, and rewrite the data into the precise grouped format required above.
 - **Benefit:** Ensures the file is **solver-ready**, scalable, and reproducible.
-

6. Example Workflow: Geo Script vs. Python API

6.1 Simple Cube Example: Geo Script Approach

This Geo Script defines the geometry and groupings but requires Python for the conformal fragmentation and export.

- **Script (Geo Script):**
 - `SetFactory("OpenCASCADE");`
 - `Box(1) = {-1, -0.5, 0, 1, 1, 1}; //Extreme Left`
 - `Box(2) = {1, -0.5, 0, 1, 1, 1}; //Extreme Right`
 - `Box(3) = {0, -0.5, 0, 1, 1, 1}; //Middle`
 -
 - `// Grouping`
 - `Physical Volume ("Middle Cube") = {3};`
 - `Physical Volume ("Left&Right Cube") = {1,2};`
 - `Physical Surface("ExternalFaces") = {1, 3, 4, ...};`
 -
 - `// Meshing Commands`
 - `Mesh (3);`

- `Mesh.ElementOrder = 1;`

6.2 Python API Logic for Conformal Meshes

The Python API provides the logic required to identify volumes accurately and prepare for conformal meshing.

- **Fragmentation Snippet (Python API):**

Python

```
# Use fragment to ensure shared nodes across the interfaces
out_dim_tags, out_dim_tags_map = gmsh.model.occ.fragment(
    [(3, left), (3, mid), (3, right)], []
)
gmsh.model.occ.synchronize() // CRITICAL: This updates the internal
model; skipping it causes silent failures.
```

- **Grouping Logic Snippet (Python API):** Uses geometric checks to ensure correct material assignment.

Python

```
# Identify volumes by checking their X-coordinate
for dim, tag in out_dim_tags:
    if dim == 3:
        mass = gmsh.model.occ.getCenterOfMass(dim, tag)
        x_center = mass[0]
        if -0.1 < x_center < 0.6:
            middle_volumes.append(tag)

# Define Final Grouping (e.g., Middle Cube)
gmsh.model.addPhysicalGroup(3, middle_volumes, tag=1)
```

7. Project Summary and Lessons Learned

- **Scripting is Mandatory:** The Python API is the control center for reproducible FEM workflows.
- **Fragmentation is Essential:** Always use `fragment` for multi-material interfaces to guarantee a conformal mesh.
- **Always Synchronize:** Missing `gmsh.model.occ.synchronize()` is the most common cause of script failure. **This command updates the internal GMSH model, preventing topology errors and silent failures in subsequent meshing and grouping steps.**
- **Post-Process:** The raw GMSH export requires a final Python script to restructure the element blocks into the Abaqus solver-ready format.